

## Call by reference

Call by reference means that the argument passed to the function is a reference to a variable that already exists in memory rather than an as an independent copy of that variable. Hence any changes made to the variable will be impacted even after the function is executed.

Dictionary passed as argument is call by reference; for example:

```
dict1 = {'a': 1, 'b': 2, 'c': 3, 'd': 4, 'e': 5}

def change(dict_val):
    print("inside change function before change:", dict_val)
    dict_val['new'] = 100
    print("inside change function after change:", dict_val)

print("inside main function before change:", dict1)
change(dict1)
print("inside main function after change:", dict1)
```

Output:

```
inside main function before change: {'a': 1, 'b': 2, 'c': 3,
'd': 4, 'e': 5}
inside change function before change: {'a': 1, 'b': 2, 'c': 3,
'd': 4, 'e': 5}
inside change function after change: {'a': 1, 'b': 2, 'c': 3,
'd': 4, 'e': 5, 'new': 100}
inside main function after change: {'a': 1, 'b': 2, 'c': 3,
'd': 4, 'e': 5, 'new': 100}
```

Let's now look at some variations.

## UserDict

Users can implement their own dictionary by extending UserDict, which works like standard dict. It is preferred to use subclass from UserDict rather than from dict, as the built-in has some implementation shortcuts that end up forcing an override, which can be easily inherited from UserDict. For example:

```
# Python program to demonstrate Userdict
from collections import UserDict

# Creating a Dictionary where
# deletion is not allowed
class MyDict(UserDict):

    # Function to stop deletion
    # from dictionary
    def __del__(self):
        raise RuntimeError("Deletion not allowed")

    # Function to stop pop from
```

```
# dictionary
def pop(self, s = None):
    raise RuntimeError("Deletion not allowed")

# Function to stop popitem
# from Dictionary
def popitem(self, s = None):
    raise RuntimeError("Deletion not allowed")

# Driver's code
d = MyDict({'a':1,
            'b': 2,
            'c': 3})

print("Original Dictionary")
print(d)

d.pop(1)
```

Output:

```
Original Dictionary
{'a': 1, 'b': 2, 'c': 3}
Traceback (most recent call last):
  File "dict_sample.py", line 154, in <module>
    d.pop(1)
  File "dict_sample.py.py", line 139, in pop
    raise RuntimeError("Deletion not allowed")
RuntimeError: Deletion not allowed
Exception ignored in: <function MyDict.__del__ at 0x10955b8b0>
Traceback (most recent call last):
  File "dict_sample.py.py", line 134, in __del__
RuntimeError: Deletion not allowed
```

## OrderedDict

This maintains the order of the insertion. The popitem method of an OrderedDict pops the first item by default, but if called by popitem(last=True) will return the last item added.

Ordered dictionary is designed to remember the order of items, which is defined by the insertion order of keys. When the value of a certain key is changed, the position of the key remains unchanged.

The syntax is OrderedDict(). Here's an example:

```
from collections import OrderedDict
numbers = OrderedDict()
numbers["one"] = 1
numbers["two"] = 2
numbers["three"] = 3

print(numbers) # OrderedDict([('one', 1), ('two', 2),
                              ('three', 3)])
```